

Procedimentos e Funções

Prof. Me. Alex Paixão

Lagarto – SE

Funções

Roteiro:

- Funções
 - Declaração e chamada
- Funções importantes
- Exemplos de funções
- Variáveis
 - Globais, locais e parâmetros

Funções

Funções:

- São um trechos de código fora do programa principal
- Implementam um subalgoritmo do programa
- São utilizadas (chamadas) em diversos pontos do programa

Funções

Vantagens:

- Organiza o código:
 - Cada função é um algoritmo
 - Dividir um programa complicado em várias funções simples
- Evita repetição de código semelhante
 - Escrever o código apenas uma vez
 - Usar a mesma função várias vezes para variáveis diferentes
- Simplifica e agiliza a programação

Funções

Exemplo: comparar a média das duas melhores notas do aluno A e do aluno B

Algoritmo:

- Ler 3 notas dos alunos A e B
- Calcular a média de A
- Calcular a média de B
- Comparar as médias
- Imprimir resultados

Funções

Alguns problemas no exemplo...

- Ler 3 notas dos alunos A e B
- Calcular a média de A
- Calcular a média de B
- Comparar as médias
- Imprimir resultados

Repetição do
mesmo algoritmo

Solução: Uma função genérica para calcular a média

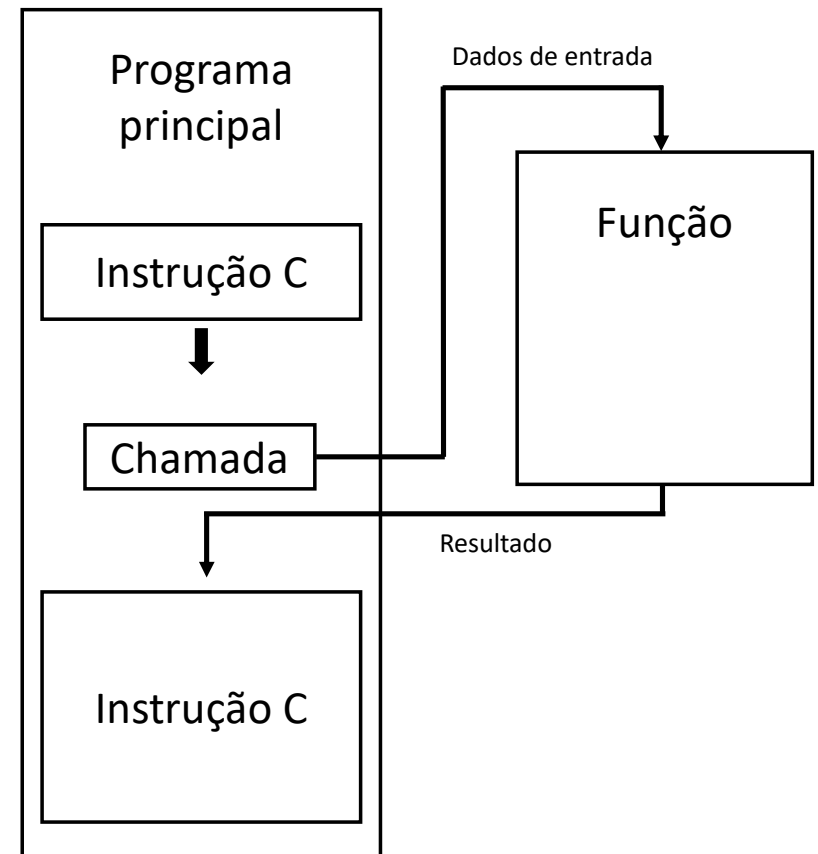
Funções

Chamadas de funções

Funções

Chamada de função:

1. Interrupção do programa principal
2. Passagem de dados de entrada para a função
3. Execução do código da função
4. Retorno do resultado
5. Continuação do programa principal



Funções

Chamada de função

Para chamar uma função:

sentença(s);

...

resultado = nome(entr1, entr2, ...);

Resultado

Nome da
função

...

sentença(s);

Valores de
entrada(parâmetros)

Funções

Exemplo: Calcular distância entre dois pontos

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    double x1, x2, y1, y2, d;
    scanf("%lf, %lf, %lf, %lf", &x1, &y1, &x2, &y2);
    d = sqrt((x1-x2)*(x1-x2) + (y1-y2)*(y1-y2));
    printf("Distância: %f", d);
```

```
return 0;
}
```

Interrompe o programa
para executar o **sqrt**



Resultado



Função



Valores de entrada
(parâmetros)



Funções

Definição de Funções

Funções

Declaração de funções:

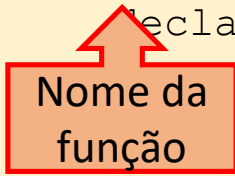
```
tipo nome (parâmetros) {  
    declaração de variáveis  
    ...  
    instruções  
    ...  
    return valor;  
}
```

Funções

Nome:

- Identifica o trecho de código da função
- O nome é usado na chamada
- Formação igual a nome de variável
- Sempre antes da função inicial

```
tipo nome (parâmetros) {  
    declaração de variáveis  
    ...  
    return valor;  
}
```



Nome da função

Funções

Parâmetros:

- Definir dados de entrada.
- São variáveis locais, declaradas e atribuídas na chamada da função.
- Lista de tipos e nomes.

```
tipo nome(parâmetros) {  
    declaração de variáveis  
    ...  
    in parâmetros  
    ...  
    return valor;  
}
```



Funções

Parâmetros

Variáveis locais, somente
existe na função



```
... media (float nota1, float nota2, float nota3) {  
    Declarações  
    Instruções  
    ...  
}
```

Funções

Parâmetros na chamada de função:

- Ao chamar a função, o valor de cada parâmetro deve ser informado
- Regras:
 - Um valor para cada parâmetro
 - Valores compatíveis com o tipo do parâmetro
 - Valores na mesma ordem que na declaração

Chamada de função:

```
Nome (parametro1, parametro2, ...)
```


Funções

Vamos Programar

Tempo de vida de uma variável

Exemplo: Declaração dentro do corpo da função
– Visibilidade apenas na função

```
int funcao(void) {  
    int v = 0;  
    ...  
}  
int main(void) {  
    ...  
    a();  
}
```

Visibilidade da variável v

Aqui a variável v não existe

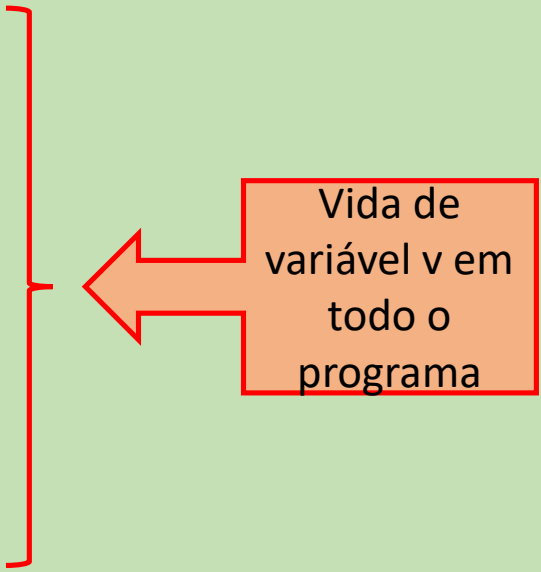
Tempo de vida de uma variável

Exemplo: Declaração dentro do corpo da função
– Visibilidade apenas na função

```
int v = 0;
```

```
int funcao(void) {  
    ...  
}
```

```
int main(void) {  
    ...  
    a();  
}
```



Vida de
variável v em
todo o
programa

Funções

Vamos Programar

Tempo de vida de uma variável

Variável Global:

```
int v = 0;
void f1(void) {
    v++;
    printf("f1: v = %d\n", v);
}
void f2(void) {
    v+=2;
    printf("f2: v = %d\n", v);
}
int main(int argc, char argv[]) {
    f1();
    f2();
    f1();
}
```

Resultado:

f1: v =

f2: v =

f3: v =

Tempo de vida de uma variável

Variável Global:

```
int v = 0;
void f1(void) {
    v++;
    printf("f1: v = %d\n", v);
}
void f2(void) {
    v+=2;
    printf("f2: v = %d\n", v);
}
int main(int argc, char argv[]) {
    f1();
    f2();
    f1();
}
```

Resultado:

f1: v = 1

f2: v = 3

f3: v = 4

Tempo de vida de uma variável

Onde escrever o código de funções

Até agora:

```
int funcaoA(int a) {  
    ...  
}
```

```
float funcaoB(char c, double r) {  
    ...  
}
```

```
char funcaoC(char c, int x) {  
    ...  
}
```

```
//  
int main( ... ) {  
    ...  
}
```

A funcaoA é conhecida aqui

A funcaoA, funcaoB são conhecidas aqui

A funcaoA, funcaoB, funcaoC são conhecidas aqui

Tempo de vida de uma variável

Código das funções no final: Programa

```
....  
int funcaoA(int a);  
float funcaoB(char c, double r);  
char funcaoC(char c, int x);  
//  
Int main( ... ) {  
    ... }  
//  
int funcaoA(int a) {  
    ... }  
float funcaoB(char c, double r) {  
    ... }  
char funcaoC(char c, int x) {  
    ... }
```



Quando declarado em cima, todas as funções são conhecidas

